Report on Side Channel Attack

Md. Tamim Iqbal
ID: 2005028

20th June 2025

1 Dataset Description

In our side-channel analysis experiments, we utilized two distinct datasets: a smaller development set containing 3,000 samples and a larger, merged dataset containing over 70,000 samples. Both datasets were collected from three popular websites: `https://cse.buet.ac.bd/moodle/`, `https://google.com`, and `https://prothomalo.com`. Each trace represents a sequence of cache-related measurements captured during website loading.

Table 1: Comparison of the Two Datasets Used

Attribute	**dataset_2005028_3k.json**	**dataset_merged_70k.json**
Total Samples	3,000	70,491
Websites (Classes)	3	3
Samples per Class	1,000 each	23,497 each
Balance Ratio (min/max)	1.000	1.000
Trace Length (avg / min / max)	1000 / 1000 / 1000	1000 / 1000 / 1000
Trace Length Median	1000.0	1000.0

Both datasets are **perfectly balanced** across classes, with each website contributing equally. The traces have a fixed length of 1000, ensuring uniform input dimensionality for model training. This uniformity helps reduce the complexity of preprocessing and model input handling.

Website-wise Trace Statistics

Each class contains identical trace statistics for both datasets:

- `https://cse.buet.ac.bd/moodle/`: average length = 1000.0 (min: 1000, max: 1000)
- `https://google.com`: average length = 1000.0 (min: 1000, max: 1000)

- `https://prothomalo.com`: average length = 1000.0 (min: 1000, max: 1000)

This consistency in trace characteristics provides a strong foundation for training and evaluating classification models effectively.

2 Experimental Setup

In this section, we describe the experimental setup used for website fingerprinting using deep learning models on two datasets: `dataset_2005028_3k.json` and `dataset_merged_70k.json`. We trained six models in total, three for each dataset. Each model varies in architecture and complexity to analyze their classification performance and generalization behavior.

2.1 Data Preprocessing

Before training, all datasets undergo a consistent preprocessing pipeline to ensure model compatibility and fair evaluation:

- **Input Vector Length:** All traces are zero-padded or truncated to a fixed length of 1000 samples.
- **Train-Test Split:** A stratified sampling strategy is used to split each dataset into training and test sets with an 80:20 ratio.
- **Normalization:** Standard score normalization (Z-score) is applied:

$$x' = \frac{x - \mu}{\sigma}$$

  where μ and σ are the mean and standard deviation computed from the training dataset. These statistics are saved and reused during real-time inference to ensure consistent scaling.
- **Label Encoding:** Website URLs are encoded into numerical class labels using `LabelEncoder`.
- **Early Stopping:** To prevent overfitting, early stopping is applied with a patience value based on validation loss.

2.2 Model Configurations

All models are trained using the following optimization and regularization strategies:

- **Input Size, Batch Size, Learning Rate:** All models share the same input size of 1000, batch size of 64, and learning rate of 1×10^{-4}.
- **Loss Function:** Cross-entropy loss is used for multi-class classification.

- **Optimizer:** Adam optimizer is used for all training sessions.
- **Early Stopping:** To prevent overfitting, early stopping is applied with a patience of 5 or 10 epochs.

The table below outlines the specific hyperparameters used in each model:

Table 2: Model Hyperparameters

Model	Dataset	Hidden Size	CNN Channels	LSTM Hidden	LSTM Layers	Epochs	Patience
Simple_3k	dataset_2005028_3k.json	128	–	–	–	50	5
Complex_3k	dataset_2005028_3k.json	128	–	–	–	50	5
CNNLSTM_3k	dataset_2005028_3k.json	–	64	128	1	50	5
Simple_70k	dataset_merged_70k.json	128	–	–	–	100	10
Complex_70k	dataset_merged_70k.json	128	–	–	–	100	10
CNNLSTM_70k	dataset_merged_70k.json	–	64	128	1	220	10

3 Results

This section presents the evaluation results for each model, including best test accuracy and the corresponding accuracy and loss curves over training epochs.

3.1 Models Trained on `dataset_2005028_3k.json`

3.1.1 Simple_3k

- **Best Accuracy:** `93.83%`

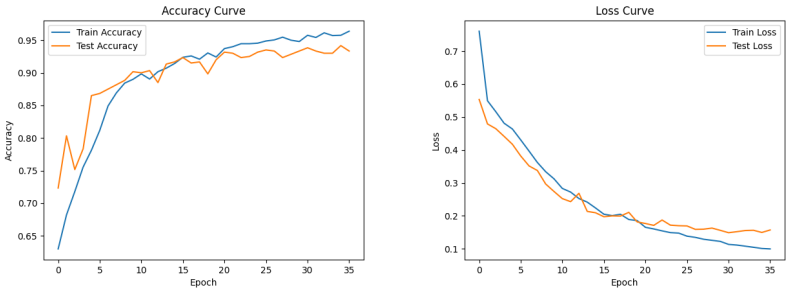

Figure 1: Training curves for Simple_3k: Accuracy (left), Loss (right)

3.1.2 Complex_3k

- **Best Accuracy:** `94.33%`

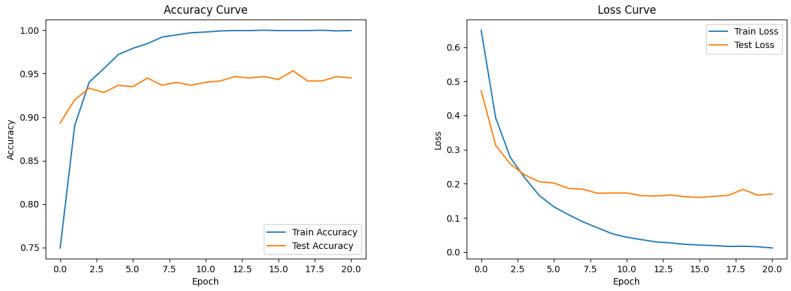

Figure 2: Training curves for Complex_3k: Accuracy (left), Loss (right)

3.1.3 CNNLSTM_3k

- **Best Accuracy:** `63.33%`

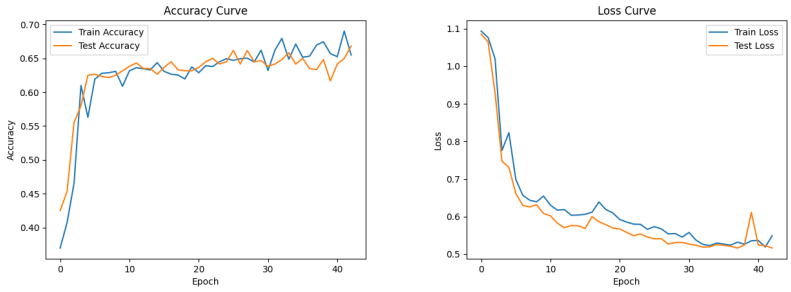

Figure 3: Training curves for CNNLSTM_3k: Accuracy (left), Loss (right)

3.2 Models Trained on `dataset_merged_70k.json`

3.2.1 Simple_70k

- **Best Accuracy:** `76.81%`

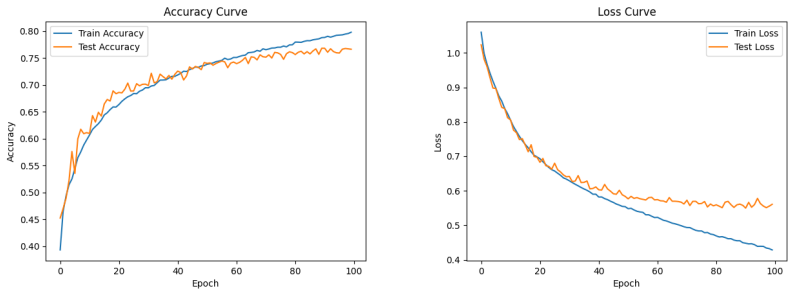

Figure 4: Training curves for Simple_70k: Accuracy (left), Loss (right)

3.2.2 Complex_70k

- **Best Accuracy:** `77.53%`

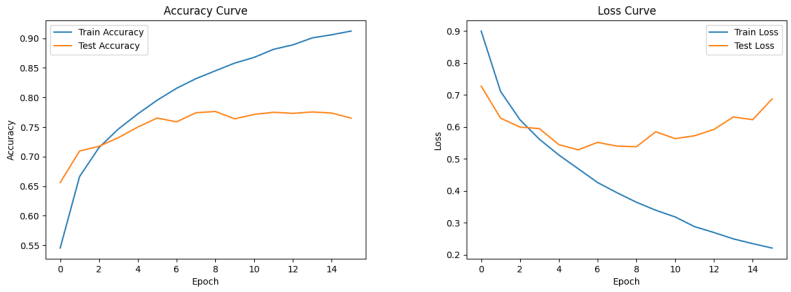

Figure 5: Training curves for Complex_70k: Accuracy (left), Loss (right)

3.2.3 CNNLSTM_70k

- **Best Accuracy:** `83.08%`

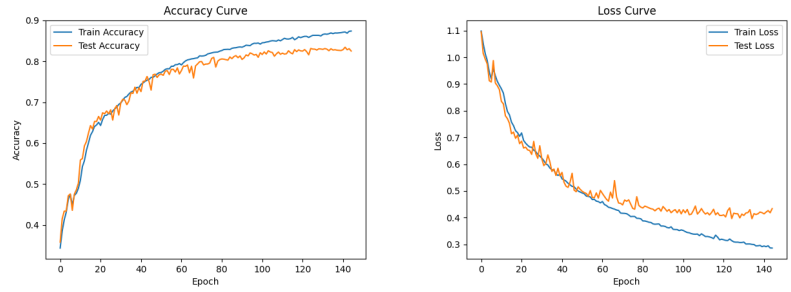

Figure 6: Training curves for CNNLSTM_70k: Accuracy (left), Loss (right)

4 Model and Dataset Access

All resources used in this project — including datasets and trained model checkpoints — are available at the following URL:

https://drive.google.com/drive/folders/1pFveHQRElsVaF-Yc4XGwQ-3rAAiIUm3h?usp=drive

This includes:

- `dataset_2005028_3k.json` and `dataset_merged_70k.json`
- Trained models: `Simple`, `Complex`, and `CNNLSTM` variants
- Preprocessing files: `scaler.pkl`, `label_encoder.pkl`

5 Discussion

The experimental results demonstrate several important observations regarding model performance, dataset characteristics, and the effectiveness of architectural choices:

- **Model Complexity vs. Dataset Size:** On the smaller dataset (`dataset_2005028_3k.json`), both the Simple and Complex models achieved high accuracy (over 93%), whereas the CNNLSTM model struggled to generalize, achieving only 63.33%. This suggests that complex models like CNNLSTM may overfit on limited data or require larger datasets to learn effectively.
- **Improvement with Larger Dataset:** On the larger dataset (`dataset_merged_70k.json`), all models showed improved generalization compared to the 3k versions, particularly the CNNLSTM model, which achieved the highest accuracy of 83.08%. This confirms that richer datasets enable more expressive models to learn meaningful patterns.
- **Best Performing Model:** Among all configurations, the `CNNLSTM_70k` model outperformed the rest, indicating that the combination of spatial feature extraction (CNN) and sequential modeling (LSTM) is particularly effective when enough training data is available.
- **Stability with Early Stopping:** The use of early stopping (with patience values of 5 or 10 depending on the dataset) proved effective in avoiding overfitting while maintaining strong performance. This strategy was crucial, especially for the larger models trained over longer epochs.
- **Consistency in Preprocessing:** Since standard score normalization was applied using statistics computed from training data and reused at inference time, consistent scaling contributed significantly to reliable performance in real-time classification.

- **Generalization Gap:** Although models performed well on test splits, real-time inference sometimes showed reduced confidence or accuracy. This may be attributed to differences in noise characteristics, sampling artifacts, or subtle shifts in runtime trace behavior that were not captured during training.

These findings highlight the importance of matching model complexity to data availability and maintaining a consistent preprocessing pipeline. In future work, data augmentation or unsupervised pretraining could be explored to further enhance robustness.